论文解读 · arXiv 2611.08318 · 2026-11

Learning the Task or Learning the Verifier? Rethinking Criteria Exposure in Embodied Self-Improvement

学的是任务，还是验证器？——重新审视具身自我改进中的“判据暴露”

Wenbo Huang, Jianxin Bi, Shunyu Yang, Ruiyang Si, Rui Ni, Qiang Wang, Shulong Jiang, Yintianrun Zhang, Feixiang Ruan, Kewei Zhang, Ye Huang, Shukai Gong, Daquan Zhou (通讯作者)

北京大学 · 新加坡国立大学

关键词： embodied agent · self-improvement · robot policy evaluation

TL;DR

一句话总结： CE 多出来的分，主要是学会了验证器

让 learner 读判分代码（Criteria Exposure, CE）能在 RoboDojo 上多拿分，但多出的分几乎全来自**指令里没写的隐藏规则**：换成只验证指令的判据就消失，换到新任务也带不走。

读判据（w/ CE）的库

51.6% → 45.0%

16 个 criteria-limited 任务，同一批 episode 改用指令忠实判据 C_faith 重判，下降 6.6 点

只看成败（w/o CE）的库

27.8% → 47.2%

同一批 episode 改用 C_faith 重判，上升 19.4 点；原本 23.8 点的差距反转为 −2.2

读 C_faith 学出的库

65.0% → 13.5%

10 个任务，换回原判据 C_orig 判分后下降 51.5 点：分数只跟着它读过的那份判据走

背景

问题根源： benchmark 的判据经常比指令要求得多

agent 收到的指令（press_by_number）

“Press the two red buttons the required number of times according to the number cards, then press the blue button to confirm.”

按数字卡片的次数按两个红按钮，然后按蓝按钮确认。

判分代码实际检查的内容

```
repeat(press(red_left), n0)
press(blue)                          指令未写
repeat(press(red_middle), n1)
press(blue)    #  恰好两次            指令未写
both_arms_home()                     指令未写
```

现有自我改进方法向 learner 暴露判据的程度（从少到多），作者统称为 Criteria Exposure （CE）

规则描述 → 公开评分表 → 数值阈值 → 形式化目标谓词 → 验证器源码

对照：真实世界的机器人只有指令，最多再拿到人或 VLM 给出的成败判断。模仿学习策略的示教本身包含这些细节，所以对它们无害；只拿到指令的 coding agent 却无从得知。

研究问题

核心问题：读了判据， agent 学会的是任务还是验证器?

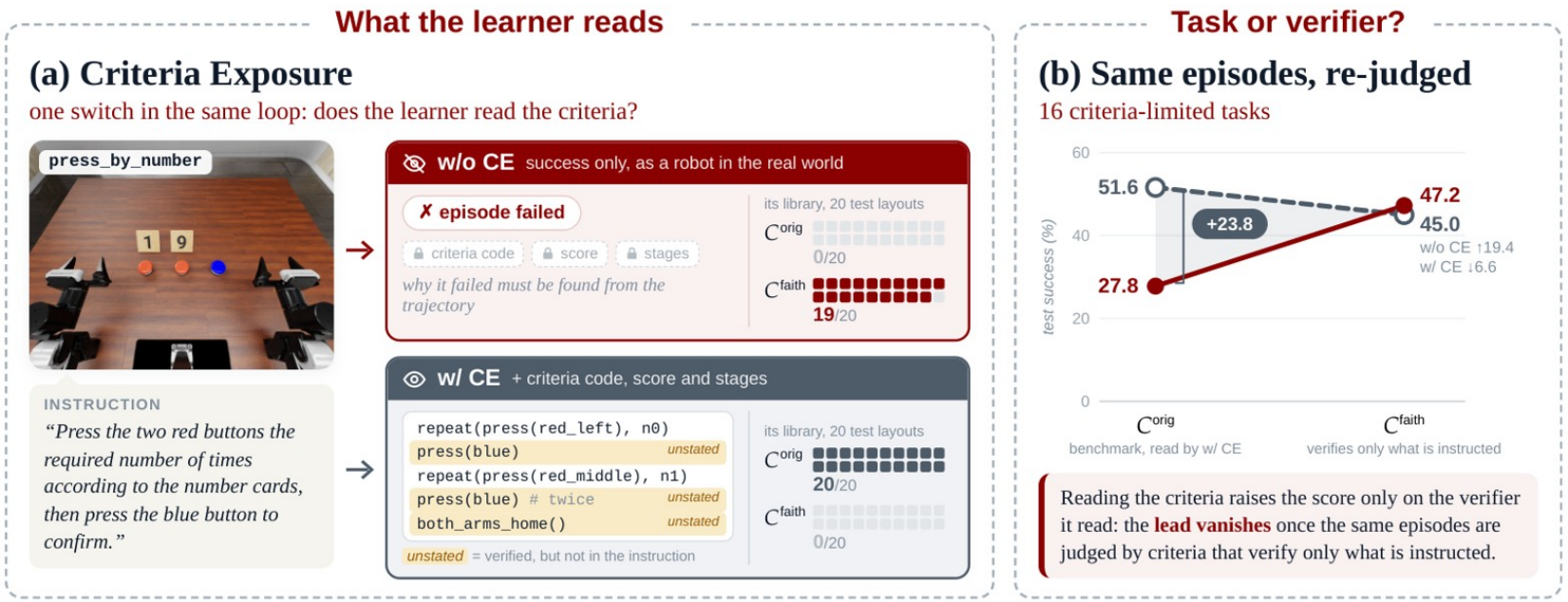


图 1 怎么读

(a) 同一个循环，只看 learner 读不读判据

- w/o CE 库： C_orig 0/20 ，C_faith 19/20
- w/ CE 库： C_orig 20/20 ，C_faith 0/20

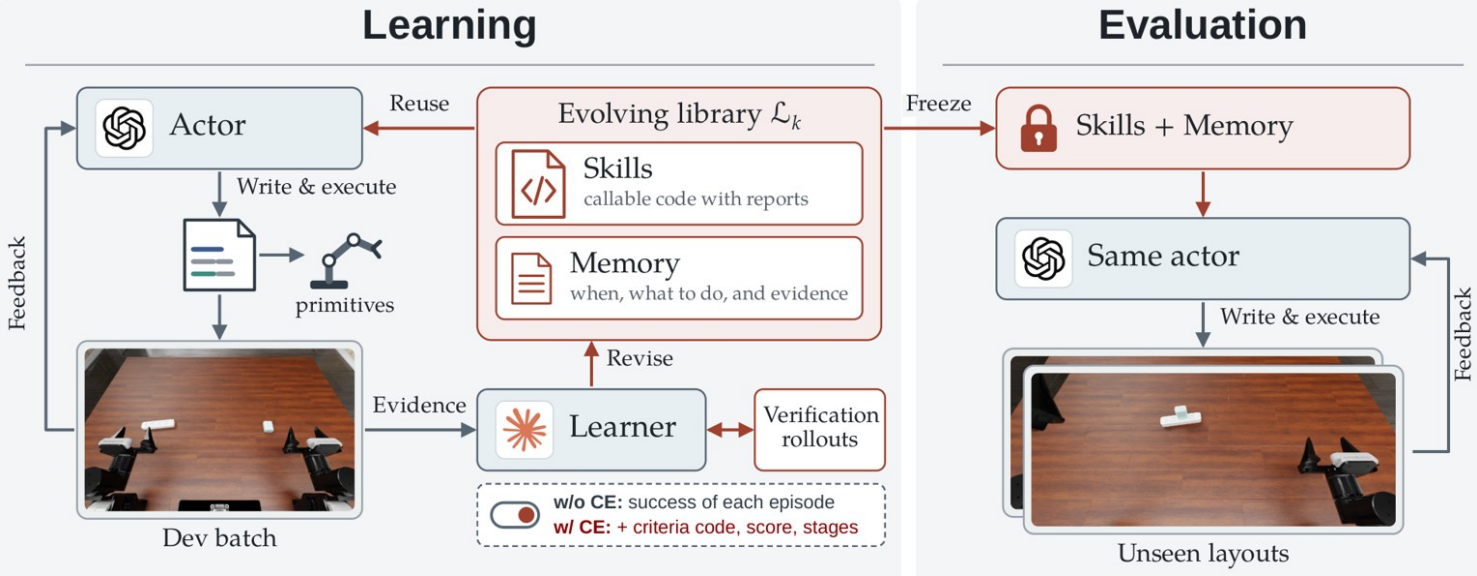
(b) 16 个任务，同一批 episode 用两套判据判

- C_orig 下 51.6 vs 27.8 ，领先 23.8
- C_faith 下 45.0 vs 47.2 ，领先消失

C_orig： benchmark 原判据；
C_faith： 作者重写、只验证指令内容的判据

方法

实验设计：同一个自我改进循环，只切换一个开关



Actor （冻结）	GPT-6 Astra，经 PyRUA-Lean primitives 写代码执行，仅用 RGB 感知
Learner	Claude Code （Opus 5.5），每轮把证据改写成一版 skill + memory 库
学习	每任务 5 轮 × 5 个 dev 布局（seed 1），明显退步就回滚
测试	库冻结只读， 20 个从未见过的布局（seed 0）

w/o CE 只看每个 episode 成功与否（及轨迹）

最接近真实世界的设定

w/ CE 另外读判据代码、 score 、 stages

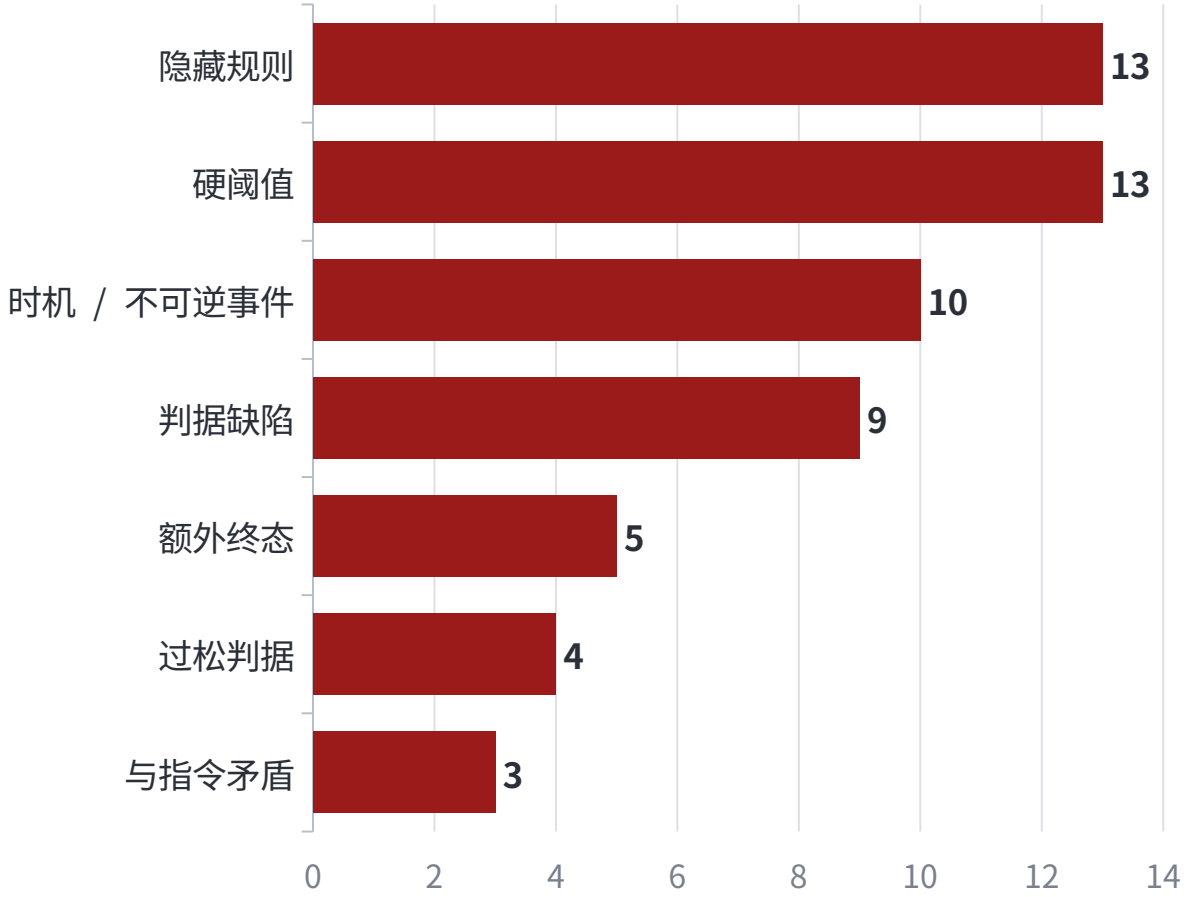
Actor 始终看不到这些

图：论文 Fig. 2；设置：论文 §3.1 、 Table 3

Criteria-limited 任务

判据审计： RoboDojo 42 个任务中有 20 个与指令不一致

各类不一致涉及的任务数（中 / 高严重度）



20 / 42

个任务判据与指令不一致（共 26 个变体），人工逐一审查确认

33 vs 5

33 个任务的判据要求双臂归位，但只有 5 条指令提到

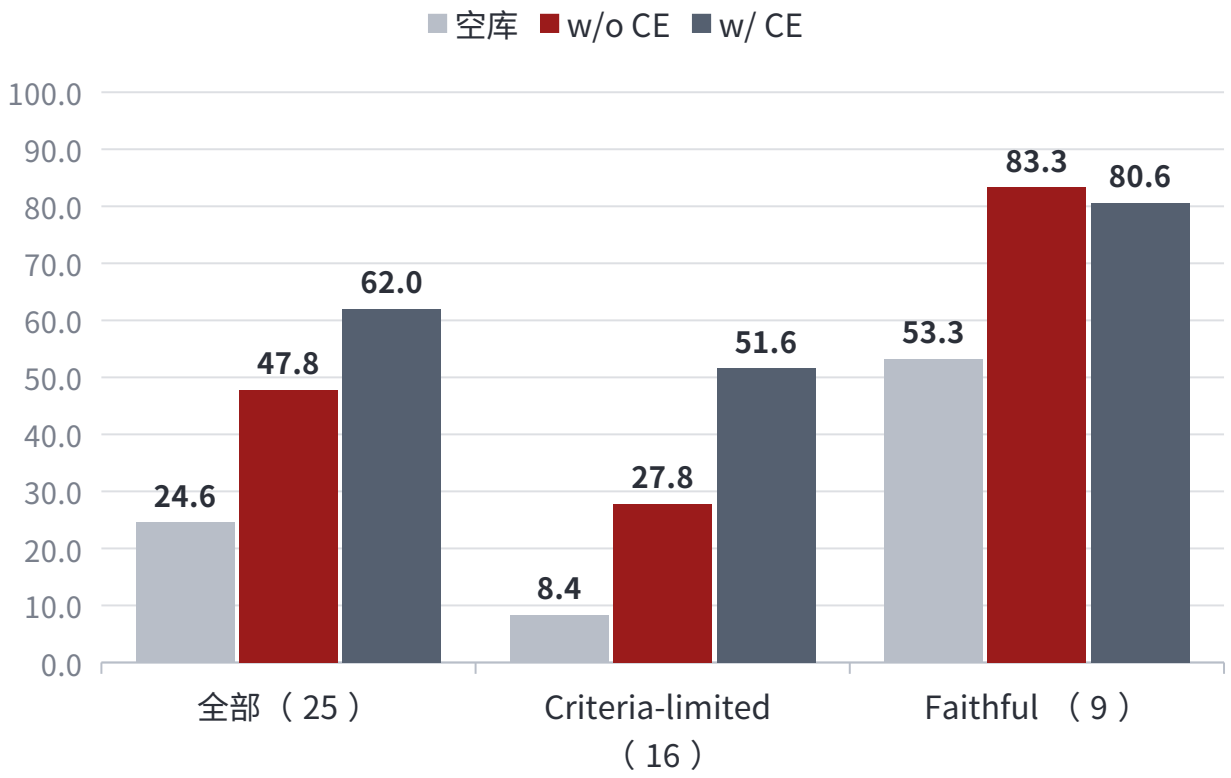
16 + 9

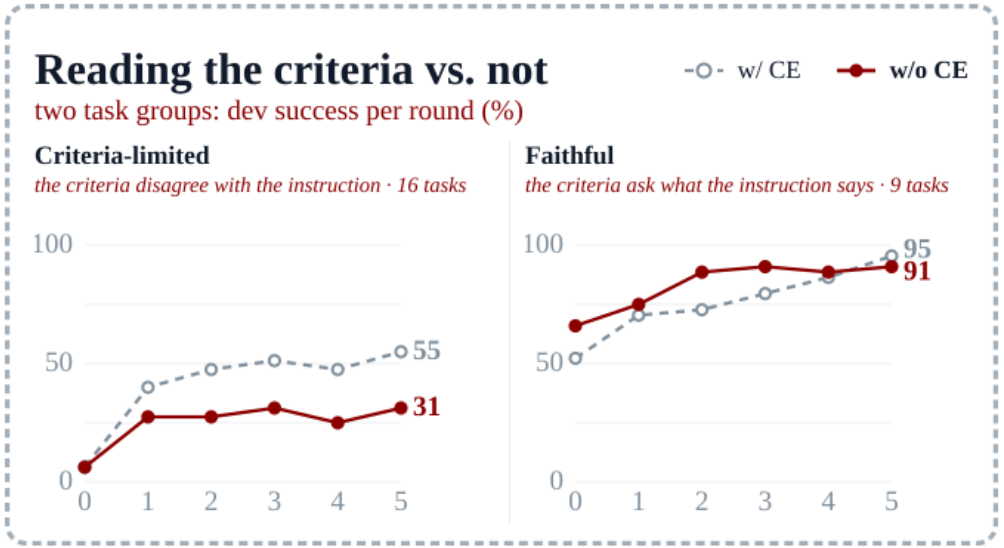
实验用 25 个任务： 16 个 criteria-limited + 9 个 faithful ，事先分组

指令忠实判据 C_faith 的写法

- 只改判分代码，不改指令；保留双臂归位
- 去掉或放宽隐藏规则，修复缺陷，放宽硬阈值
- 仿真状态能可靠区分时，收紧过松判据
- 结论翻转的 episode 逐一人工核对，未发现误判

H1：读判据的额外收益只出现在判据与指令不一致的任务上





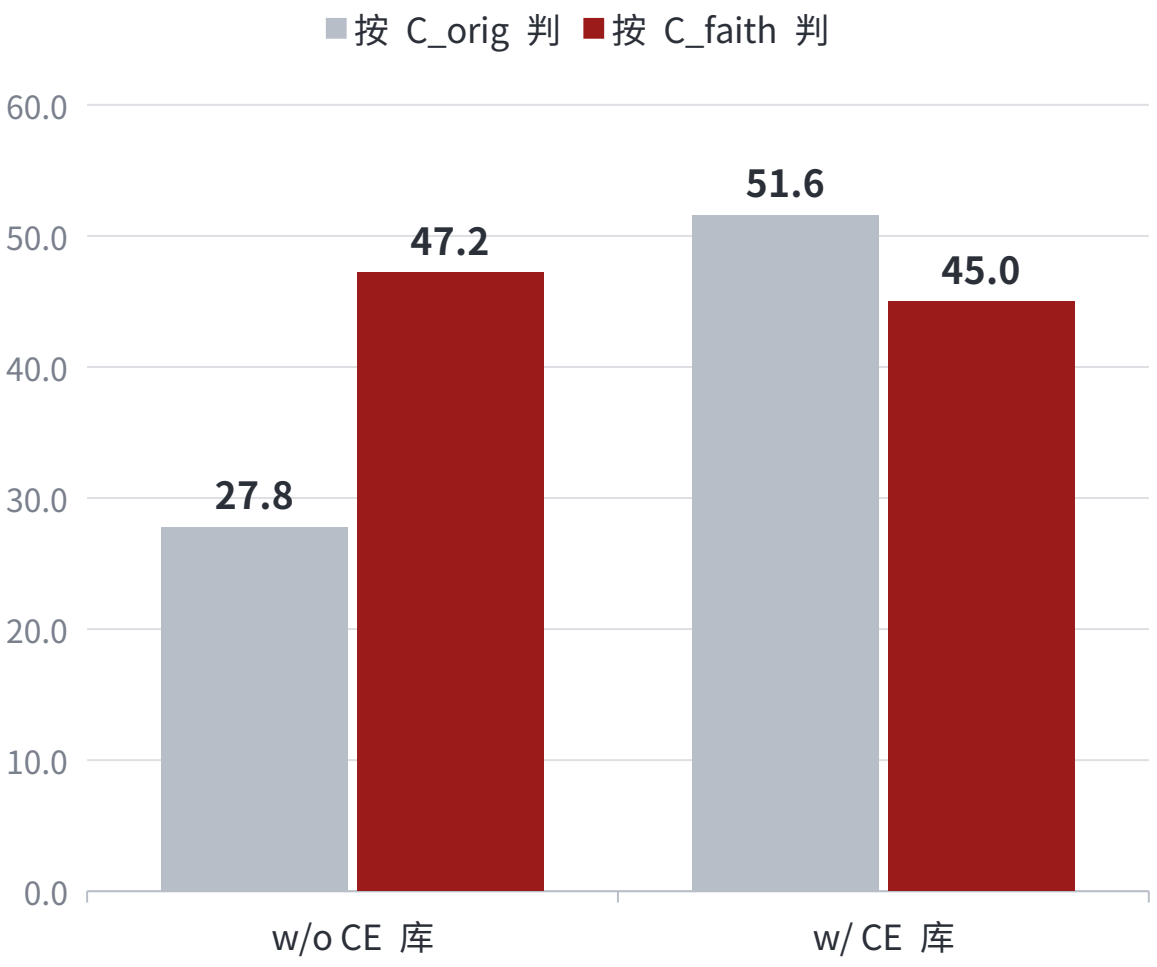
只看成败已让成功率翻倍（24.6→47.8）；CE 多出的 **14.2 点**全部来自 criteria-limited 任务，faithful 任务上两者持平。

Dev 成功率随轮次：criteria-limited 组第 1 轮就分开（40 vs 28），最终 55 vs 31；faithful 组两者都超过 90%。

- **build_tower**：剩下的方块会暴露规则，两者都 20/20
- **press_by_number**：按钮协议在失败轨迹里看不出来，只看成败的 learner 一直没找到

H2 · 换用于评判的判据（16 个任务 × 20 布局，SR %）

H2：同一批 episode 改用指令忠实判据重判，CE 的领先消失



+23.8 → −2.2

CE 的领先差距缩小 25.9 点（按任务 bootstrap，95% CI 4.7–54.4）

93 : 17 → 47 : 54

配对布局中“仅 CE 成功”对“仅 w/o CE 成功”的数量，换判据后落在噪声范围

87 / 89

w/o CE 在 C_orig 下的 89 次成功，C_faith 只否掉 2 次

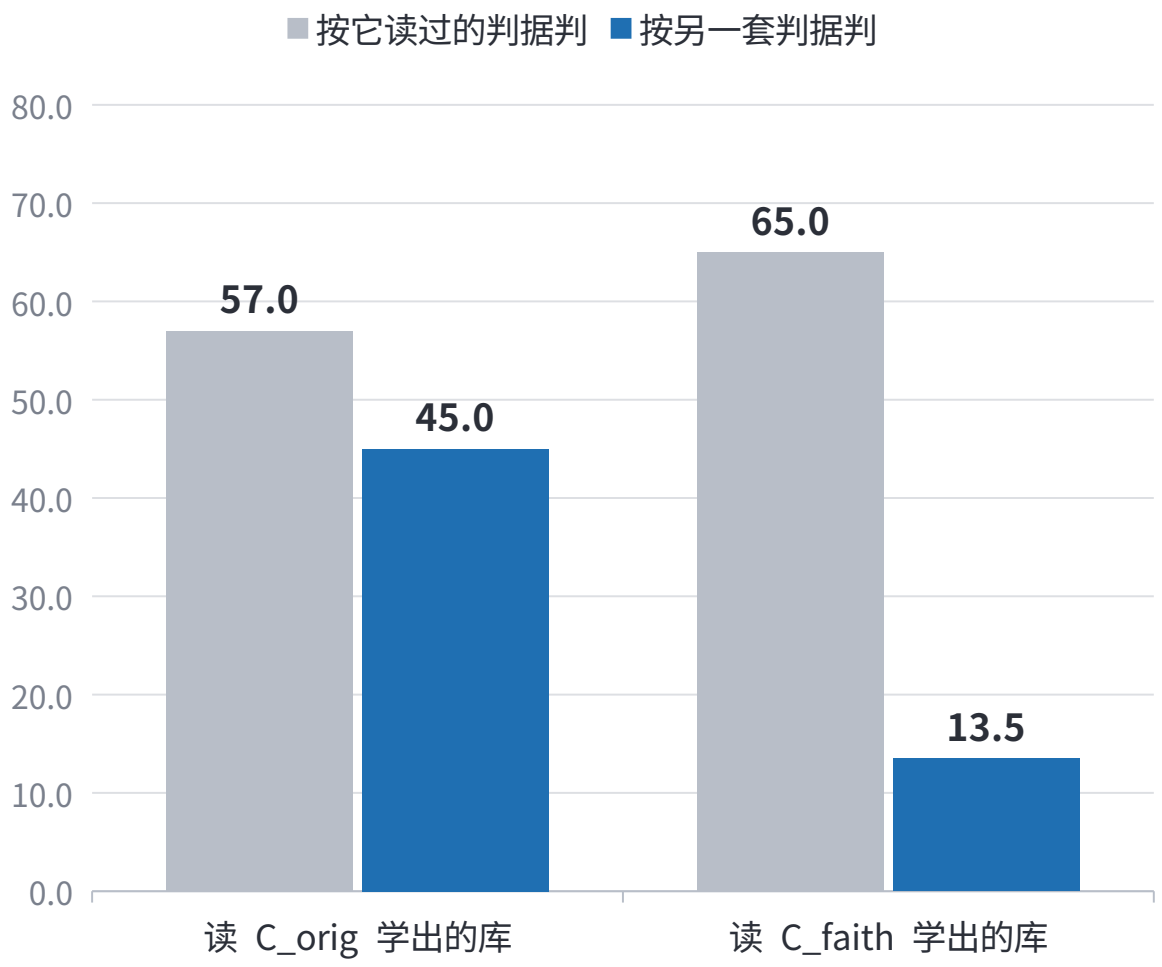
20/20 → 0/20

press_by_number 上的 CE 库；w/o CE 库则从 0/20 变成 19/20

数据：论文 §4.2.1、Fig. 1b / Fig. 5（左）

H2 · 换 learner 读的判据（10 个任务， SR %）

H2： 每个 CE 库只在自己读过的那份判据下得高分



−12.0 vs −51.5： 为什么不对称?

C_faith 以去掉、放宽规则为主。 C_orig 库换到 C_faith ，相当于标准变松； C_faith 库换回 C_orig ，则要面对一整套从没见过的规则。

即使读的是更宽松的判据，分数也跟着它走

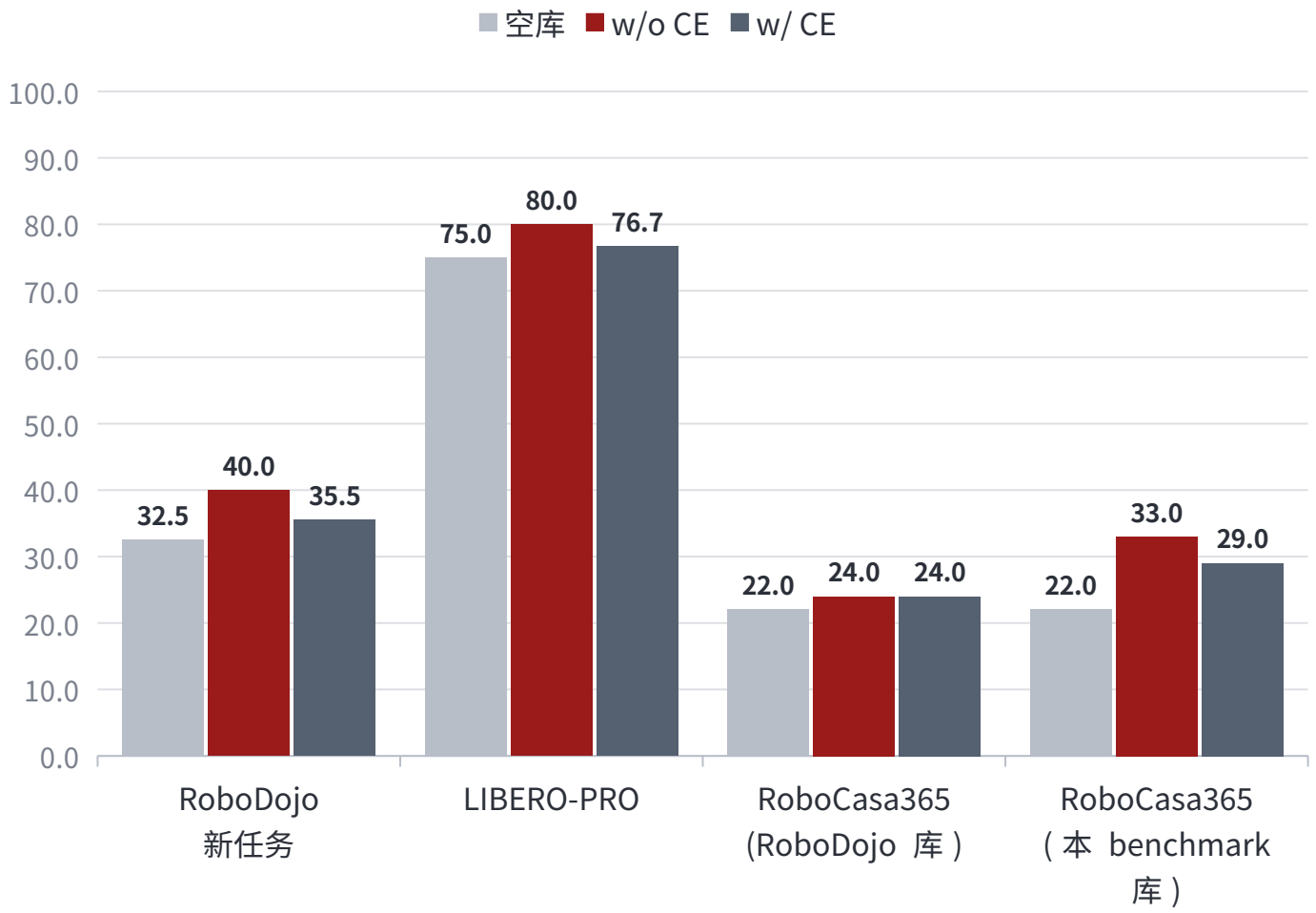
pour_by_language： 读 C_faith 的库不再把瓶子扶正（C_orig 要求），结果 C_faith 下 19/20， C_orig 下 0/20。

CE 也改变了库里写下的经验

约 **1/3** 的 CE memory 条目直接引用判据代码， w/o CE 一条都没有；只看成败的 learner 必须找物理原因。

数据：论文 §4.2.2–4.2.3 、 Fig. 5 （右）

H3： CE 的优势带不到新任务；跨 benchmark 只有目标内经验有用



目标（库来源）	空库	w/o CE	w/ CE
RoboDojo （RoboDojo）	65/200	80	71
LIBERO-PRO （RoboDojo）	45/60	48	46
RoboCasa365 （RoboDojo）	22/100	24	24
RoboCasa365 （RoboCasa365）	22/100	33	29

- **RoboDojo 内迁移到 10 个新任务：** w/o CE 反而更好（+15 vs +6），CE 在源任务上 14.2 点的领先没有再出现
- **跨 benchmark：** RoboDojo 库在两个新 benchmark 上几乎没有提升
- **解释：** CE 经验装的是“通过某个判据所需的条件和精度”，换个场景就成了噪声；夹爪之类的物理规律才能通用

结论与建议

启示：评测自我改进时， learner 不该看到验证器

01

CE 增益反映 benchmark 缺陷

CE 只在指令与判据不一致处提分，增益大小随这一差距增长，衡量的是 benchmark 的缺陷而不是 agent 的能力。

02

默认不暴露判据

learner 只拿真实机器人也能拿到的反馈（成败），并同时报告在指令忠实判据下的成功率；换判据后还在的增益，才能指望部署时保留。

03

判据要验证的写进指令

Benchmark 应在指令中写明次数、顺序、终态、双臂复位，并发布带版本号的判据和“判据与指令对照表”。

另一面：只看成败的自我改进是真实的，成功率翻倍，换判据后依然成立。

个人评价

评价：实验设计干净有力，但忠实判据和样本量仍需进一步验证

亮点

- 切中评测规范：把“ learner 读不读判据”从实现细节变成需要单独控制的实验变量
- 单开关受控实验： actor 、 primitives 、轮数、布局全部固定
- 同一条轨迹用两套判据判，把行为差异和判据差异彻底分开
- 换评判判据、换被读判据两个方向的反事实互相印证
- 附带的 42 个任务判据审计本身就是对社区有用的产出

局限

- C_faith 由作者编写且以放宽为主，部分决定带有主观性（如“什么算一座塔”），还需独立复核
- 每任务只有 20 个测试布局，汇总区间宽（ 4.7–54.4 ）；未估蒸馏方差，机制解释没有做消融
- 只比较了不暴露和给源码两个极端，中间档位没有逐级测量
- 只用了一组 actor/learner ；判据只在 RoboDojo 重写过；仿真的成败信号比真实世界干净
- 官网已公开部分规则，算不算合理的任务知识存在争议

“A robot in the real world never sees its verifier; the agent we measure should not either.”

真实世界的机器人看不到自己的验证器，我们评测的 agent 也不该看到。

—— 论文 §6 Conclusion

arXiv 2611.08318 · 项目页：dagroup-pku.github.io/PyRUA-Lean/RSIJudge